Online Order Processing with the Facade Pattern

Assignment 4 | Software Design Patterns | Java 17

Course ShP-2216 | Option B | GitHub owner bossmerej-ship-it

1 Introduction

This project models a small bookshop. A customer can place an order, cancel the current order, and view its status. Checkout needs four services: inventory, payment, shipping, and notifications. A facade puts these steps behind one simple call, so the console client does not have to coordinate them itself. This matches the Facade pattern [1].

The implementation contains only six Java files: Main, OrderFacade, InventoryService, PaymentProcessor, ShippingService, and NotificationService. There are no external Java libraries, extra data classes, or launch scripts.

2 Subsystem and workflow

InventoryService starts with five books. It checks quantity, reserves stock, calculates the price, and releases a reservation. **PaymentProcessor** records a successful payment or declines it, and can refund the recorded amount.

ShippingService checks whether the city is supported and stores the scheduled destination. **NotificationService** formats and stores the latest confirmation or cancellation message. Services retain state, so their effects can be inspected.

OrderFacade first reserves books, then requests payment, then schedules delivery, and finally prepares a notification. A declined payment releases the books. If delivery fails, the facade refunds payment and releases the books. Cancellation cancels delivery, refunds payment, and restores stock.

```
OrderFacade shop = new OrderFacade();
shop.placeOrder(2, true, "Almaty");
shop.showStatus();
shop.cancelOrder();
```

3 Scope and execution

The shop supports one active order at a time. Each book costs 12,500 whole KZT; integer arithmetic is sufficient for at most five books. Payment approval is a boolean simulation. The supported cities are Kyzylorda, Almaty, and Astana. All data is kept in memory; no actual payment, delivery, or email is sent.

Open the repository in IntelliJ IDEA, select JDK 17, mark src as Sources Root, and run Main.java. The menu offers Place order, Cancel order, Show status, and Exit.

4 UML class diagram

Main uses only OrderFacade. The facade has private references to four public services. Each service works independently and has no reference back to the facade.

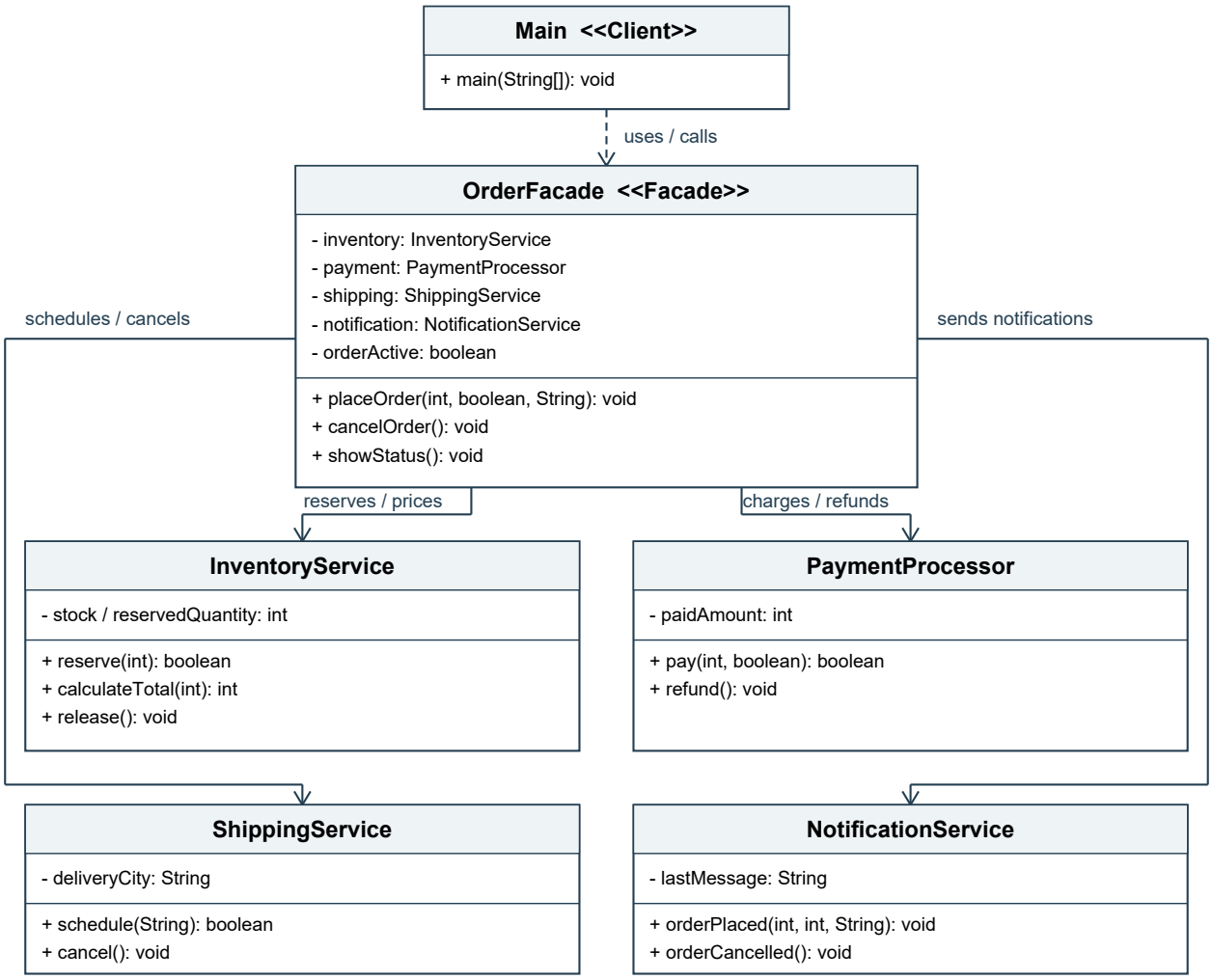


Figure 1. Dashed arrow: dependency (uses). Solid arrow: navigable association (holds a reference). + public; - private. Getters omitted.

How to read the diagram

Arrows point from the caller to the class it uses. The dashed Main-to-OrderFacade arrow means the client calls the facade. The four solid facade-to-service arrows mean the facade holds references to, and calls, those services. The labels describe each service's role. They do not show inheritance or the order of execution.

Demonstration and verification

Two books cost 25,000 KZT and reduce stock from 5 to 3. Cancellation restores stock to 5 and paid amount to 0. Payment option 2 demonstrates a decline; an unsupported city such as London demonstrates failure after payment. In both cases, final stock is 5 and the paid amount is 0.

Validation used JDK 17 with --release 17 -Xlint:all -Werror. Nine console scenarios passed, including success, cancellation, repeated cancellation, rollback, invalid quantity, and a second active order. Direct service use and repeated cleanup were also checked separately.

5 Clean Code principles

Each excerpt below is from the simplified implementation and includes its reason.

1 Meaningful names

The facade uses clear business verbs. Main asks for an order rather than managing stock or delivery details.

```
shop.placeOrder(quantity, payment.equals("1"), city);
shop.cancelOrder();
```

2 Single Responsibility

The facade coordinates while services own their rules. These calls delegate reservation and pricing to InventoryService.

```
if (!inventory.reserve(quantity)) { return; }
int total = inventory.calculateTotal(quantity);
```

3 Encapsulation

Service state is private. Other classes read payment state through a getter instead of changing it directly.

```
private int paidAmount = 0;
public int getPaidAmount() { return paidAmount; }
```

4 Proper error handling and rollback

If the courier rejects delivery, the facade reverses the earlier successful steps. It does not leave a charge or reservation behind.

```
if (!shipping.schedule(city)) {
    payment.refund();
    inventory.release();
    return;
}
```

5 Guard clauses before state changes

Inventory checks the quantity and current reservation first. Invalid input returns immediately, keeping stock unchanged.

```
if (quantity <= 0 || quantity > stock || reservedQuantity > 0) {
    System.out.println("Invalid quantity, insufficient stock, or an existing reservation.");
    return false;
}
```

6 Conclusion

OrderFacade hides service creation, call order, and rollback. Main remains a small console client. The four public services still work directly without the facade. The design meets the assignment with a limited, easy-to-explain bookshop simulation. Persistence, multiple simultaneous orders, real integrations, and concurrency are outside scope.

Repository and references

https://github.com/bossmerej-ship-it/SDP

[1] Refactoring.Guru. Facade. https://refactoring.guru/design-patterns/facade.

Requirements: Assignment_4_Facade_Pattern_Instructions.docx, supplied assignment brief. The supplied smart-home repository was used as a reference for project size and structure.